

1-3 · 이미지·컨테이너와 도커 사용하기 · 실습 1H

# 볼륨과 포트로 데이터 다루기

컨테이너는 갈아 끼워도 데이터는 볼륨에 남는다

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

# 1시간 뒤에는

---

- ① postgres:16 을 -e--p--v 와 함께 띄우고 psql 로 접속한다
- ② docker logs · pg\_isready 로 준비 완료를 확인한다
- ③ 컨테이너를 지웠다 다시 만들어도 데이터가 남는 것을 본다
- ④ 볼륨 없이 띄우면 사라지는 것도 직접 확인한다

오늘의 핵심 실험 — 4주 MinIO · 6주 Kafka · 7주 Qdrant 가 전부 이 성질 위에 선다

# 다섯 단계 — pg 는 stop 만, pgdata 는 남긴다

컨테이너가 바로 죽으면 언제나 docker logs 부터

## ① 볼륨과 함께 띄우기

```
docker run ... -v  
pgdata:...  
postgres:16
```

## ② 데이터 넣기

```
heredoc 으로 psql 에  
CREATE·INSERT
```

## ③ 지웠다 다시 만들기

```
rm 뒤 같은 볼륨으로  
run → 두 행 그대로
```

## ④ 볼륨 없이 해 보기

```
pg_novol (5433) →  
표가 사라진다
```

## ⑤ 정리

```
pg_novol 삭제 · pg 는  
stop 만
```

# docker run 한 줄 해부 — 옵션 다섯 개

|                                           |                              |
|-------------------------------------------|------------------------------|
| <b>-d</b>                                 | 백그라운드 실행                     |
| <b>--name pg</b>                          | 컨테이너 이름                      |
| <b>-e POSTGRES_PASSWORD=engineer123</b>   | 환경변수 — 이 값이 없으면 컨테이너가 바로 죽는다 |
| <b>-p 5432:5432</b>                       | 내 5432 → 컨테이너 5432           |
| <b>-v pgdata:/var/lib/postgresql/data</b> | 이름 볼륨을 데이터 디렉터리에 연결          |
| <b>postgres:16</b>                        | 이미지 — 태그를 명시                 |

postgres 이미지는 POSTGRES\_PASSWORD 를 반드시 요구한다 — 없으면 바로 Exited (1)

# 뒀는지 네 가지로 확인한다 — 처음엔 initdb 로 3초쯤

1

**docker logs pg | tail -n 5**

「database system is ready to accept connections」

2

**docker exec pg pg\_isready -U postgres**

accepting connections 가 나올 때까지 몇 초 기다렸다 다시

3

**docker volume ls**

pgdata 가 보인다

4

**docker exec -it pg psql -U postgres -c "SELECT version();"**

PostgreSQL 16.x 문자열

Mountpoint /var/lib/docker/volumes/pgdata/\_data 는 Desktop 의 리눅스 VM 안 — 탐색기·파인더로는 못 연다

# 직접 타이핑은 -it, 흘려 넣기는 -i

## -it — 직접 타이핑

```
docker exec -it pg psql -U postgres
```

- -i 표준 입력 열기, -t 터미널 붙이기
- psql 프롬프트에서 직접 친다

VS

## -i 만 — heredoc 흘려 넣기

```
docker exec -i pg psql -U postgres << 'EOF'
```

- CREATE TABLE sensor ... INSERT ... SELECT
- -t 를 붙이면 「stdin is not a terminal」로 실패
- 확인 — temp 21.5 · humidity 48.2 두 행

# 컨테이너를 완전히 지웠다가 같은 볼륨으로 다시 만든다

1

## 완전히 삭제

`docker stop pg → docker rm pg → ps -a` 에 pg 없음 · `volume ls` 에 pgdata 는 남음

2

## 같은 볼륨으로 새 컨테이너

1단계와 같은 `docker run ... -v pgdata:... → sleep 3` (바로 치면 준비 중이라 실패)

3

## 아까 데이터가 있는가

`SELECT * FROM sensor;` → temp·humidity 두 행 그대로

뜻

컨테이너 교체  
데이터는 볼륨에

# 반대로, 볼륨 없이 하면 (pg\_novol · 포트 5433)

## 이름 볼륨 pgdata

-v pgdata:/var/lib/postgresql/data



## -v 없이

postgres 이미지가 만든 익명 볼륨 — 새로 만들 때마다 새것이 붙는다

VS



# 이것이 보여야 성공, 이것은 치지 말 것

---

- 1 -p 5433:5432 — 바꾼 것은 내 쪽 번호뿐, 안쪽은 이미지 몫
- 2 마지막 SELECT 가 relation "t" does not exist 여야 정상
- 3 익명 볼륨(긴 해시)은 volume prune — pgdata 는 안 지워짐
- 4 pg 는 rm 말고 stop 만 — 1-5 에서 5432 를 다시 쓴다
- 5 docker volume rm pgdata 는 치지 말 것 — 되돌릴 수 없다

1-3 · 실습 1H 마무리

## 다음은 정리·회고 (0.5H)

오늘 쓴 명령을 한 장으로 모으고, 남은 컨테이너를 1-4 실습을 위해 정돈합니다

✓ 도커 명령 요약 표 한 장

✓ `docker ps -a · volume ls · images` 로 정돈 확인

✓ 자주 겪는 오류 세 가지

✓ 1-4 Dockerfile 예고

지금 상태

pg 는 Exited · pgdata 볼륨은 그대로